

Rapport de projet — GENET-X

Sommaire :

L'équipe de GENET-X.....	3
Les membres.....	3
Les rôles.....	3
Cahier des charges.....	3
Le concept du jeu.....	3
Les caractéristiques du jeu.....	4
Type de jeu.....	4
Caractéristiques générales du jeu.....	4
Caractéristiques graphiques.....	4
Caractéristiques sonores.....	4
Site internet.....	4
Communication.....	5
Sessions de travail.....	5
Le jeu.....	6
Interface utilisateur et direction artistique.....	6
La carte.....	7
Affichage de la carte.....	7
Passer d'île en île.....	10
Interactions dans les grottes.....	10
Le joueur.....	11
Création et intégration des graphismes des personnages.....	11
Mise en place de la classe Player.....	12
Gestion des attributs et des animations.....	12
Développement des déplacements des joueurs.....	13
Synchronisation entre déplacements et animations.....	13
Organisation du projet et intégration finale.....	14
Affichage du joueur distant.....	14
Réseau.....	15
Objectif initial.....	15
Détection automatique des serveurs.....	15
Compatibilité multiplateforme.....	16
Amélioration du système de détection réseau.....	16
Validation de l'identité des serveurs.....	16
Limitation du nombre de joueurs.....	17
Intégration réseau dans le jeu.....	17
Architecture TCP / UDP.....	17
Gestion des déconnexions.....	18
Quêtes.....	18
Résolution.....	18
Première quête.....	19

25.05.2026

v. 3.0

Deuxième quête.....	20
Troisième quête.....	20
Quatrième quête.....	21
Cinquième quête.....	21
Sixième quête.....	22
Sauvegarde d'une partie.....	23
Fonctions réutilisables pour l'affichage.....	24
Lien avec le serveur et intégration du personnage.....	25
Items.....	26
Musiques.....	26
Cinématiques.....	27
BOTS.....	28
Intégration des bots.....	28
Organisation avec le BotManager.....	29
Catégories de bots et progression dans le jeu.....	30
Exécutable.....	31
Site internet.....	32
Ce que nous aurions aimé faire.....	32
Conclusion.....	32
Annexes.....	32
Sites.....	32
Intelligence Artificielle.....	32

L'équipe de GENET-X

Les membres



Mattéo L.

Mathieu C.

Kylian D.

Charlie M.

Victor G.

Les rôles

Responsable Connexion LAN :	Mattéo L.
Responsables Cinématiques :	Mattéo L.
Responsable Sauvegarde d'une partie :	Mattéo L.
Responsable Menu de démarrage :	Mathieu C.
Responsable Graphismes joueurs :	Mathieu C.
Responsable Création d'un exécutable :	Mathieu C.
Responsable Création des équipements utilisables :	Kylian D.
Responsable Hitbox :	Kylian D.
Responsable Déplacement joueurs :	Kylian D.
Responsable Exploitation de la génétique dans le gameplay :	Charlie M.
Responsable Limites de la carte :	Charlie M.
Responsable Site internet :	Victor G.
Responsable Graphismes cartes :	Victor G.
Responsable Création des énigmes :	Victor G.

Cahier des charges

Le concept du jeu

GENET-X est un jeu multijoueur jouable uniquement en duo. Nos deux joueurs incarnent un et une scientifique envoyés sur un archipel ravagé par une pandémie. Les joueurs doivent profiter de leur génétique différente pour accéder à des îles où ils sont les seuls à survivre. En résolvant des énigmes, les scientifiques reçoivent des indices sur la nature de la pandémie et de nouvelles îles deviennent accessibles. Nos joueurs doivent accéder à la dernière île où ils trouveront une solution pour sauver l'archipel.

25.05.2026

v. 3.0

Les caractéristiques du jeu

Type de jeu

GENET-X est un jeu RPG (role-play game) répondant aux caractéristiques de puzzle, réflexion, aventure et simulation.

Caractéristiques générales du jeu

Nous intégrerons de l'intelligence artificielle dans les monstres à combattre, ou encore dans un calculateur de chemin le plus court entre un point A et un point B. Le jeu se déroule en coopération en duo.

Afin de pouvoir jouer en multijoueur, GENET-X a configuré une connexion LAN.

Caractéristiques graphiques

Le jeu est dans un style pixelisé 2D vue du dessus.

Les personnages sont entièrement créés par GENET-X.

Les cartes découlent d'assets trouvés sur internet, puis customisés.

Caractéristiques sonores

Les musiques ont chacune été trouvées sur internet.

Site internet

Le site internet a été entièrement créé par nos soins.

Il est actuellement hébergé sur un dépôt GitHub public. Nous avons utilisé la fonctionnalité Github Pages afin de rendre le site internet accessible depuis n'importe quel navigateur web.

Communication

Tout au long du projet, nous avons mis en place une communication régulière afin d'assurer un bon suivi de l'avancement et une coordination efficace entre les membres du groupe.

Nous organisions des réunions périodiques, généralement un week-end sur deux ou trois, afin de faire le point sur l'état d'avancement des différentes tâches. Ces réunions permettaient notamment d'identifier les éventuels besoins d'aide, de discuter des problèmes rencontrés, de définir les prochaines étapes du projet ainsi que de fixer les échéances à venir.

En complément, nous avons utilisé un serveur Discord comme principal outil de communication quotidienne. Celui-ci nous permettait d'échanger rapidement, de poser des questions, de nous organiser et de partager diverses informations liées au projet.

Concernant la gestion du code source et la mise en commun de notre travail, nous avons utilisé GitHub à travers des dépôts privés. Un premier dépôt a été créé en début de projet, mais notre manque d'expérience dans la gestion des branches l'a rapidement rendu difficile à maintenir et peu structuré. Nous avons donc décidé de repartir sur un second dépôt privé, organisé de manière plus rigoureuse, que nous avons conservé jusqu'à la fin du développement.

Enfin, un canal dédié du serveur Discord était réservé aux notifications GitHub. Grâce à un système de webhooks, chaque nouveau push effectué sur le dépôt générait automatiquement une alerte sur Discord. Cela permettait à tous les membres de rester informés des modifications récentes et de savoir lorsqu'une mise à jour locale était nécessaire.

Sessions de travail

En complément de notre communication régulière, nous avons organisé à plusieurs reprises des sessions de travail en présentiel, généralement en petits groupes.

Ces séances de travail collaboratif ont contribué à améliorer notre productivité en instaurant un cadre propice à l'avancement du projet. Elles nous permettaient notamment de nous concentrer sur des fonctionnalités nécessitant l'implication de plusieurs membres, de coordonner plus efficacement certaines tâches techniques,

25.05.2026

v. 3.0

mais également d'obtenir rapidement des retours, des avis extérieurs ou une aide immédiate face aux difficultés rencontrées.

Ces moments de travail partagé ont ainsi favorisé une meilleure collaboration et accéléré la résolution de certains problèmes.

Le jeu

Interface utilisateur et direction artistique

Une partie importante de notre travail a porté sur l'interface utilisateur du jeu. Au départ, l'objectif n'était pas seulement d'ajouter des boutons ou des textes à l'écran, mais de donner au projet une identité visuelle plus cohérente et plus facile à reconnaître. Comme GENET-X est un jeu en pixel art, avec une ambiance à la fois scientifique, futuriste et liée à la génétique, il fallait éviter une interface trop générique. L'interface devait accompagner l'univers sans prendre le dessus sur la carte, les personnages et les quêtes.

Pour cela, nous avons commencé par rechercher une direction artistique cohérente. Nous avons utilisé des outils de palette de couleurs comme Colors afin de trouver une combinaison qui fonctionne à la fois pour les menus, les écrans de quête, les éléments importants et les retours d'action. Le choix s'est orienté vers une palette assez contrastée, composée notamment d'un bleu profond, d'un rose vif, de jaune, de noir et de blanc. Le bleu permet de conserver une impression technologique et scientifique et surtout de rappeler la couleur de l'océan, tandis que le rose sert de couleur d'accent pour les actions importantes ou les boutons principaux. Le jaune est utilisé pour le fond des différentes vues et surtout parce qu'il rappelle le jaune du sable d'une île. Cette répartition rend l'interface plus lisible, car chaque couleur possède un rôle facilement identifiable.

Ce travail sur la couleur avait aussi un intérêt pratique. Dans un jeu coopératif, les joueurs doivent comprendre rapidement ce qu'ils peuvent faire, où ils doivent cliquer et quel élément de l'écran leur donne une information importante. Une palette incohérente aurait rendu les menus plus difficiles à lire, surtout lorsque plusieurs interfaces apparaissent au cours des quêtes. L'idée était donc d'obtenir une interface simple, mais suffisamment marquée pour rester identifiable tout au long du jeu.

Nous avons également travaillé sur la hiérarchie visuelle. Les titres, les textes d'information, les boutons et les indications contextuelles ne doivent pas avoir le même poids à l'écran. Les titres servent à situer le joueur dans le menu ou la quête. Les textes d'aide donnent une consigne ponctuelle. Les boutons doivent

immédiatement apparaître comme des éléments interactifs. Cette hiérarchie est particulièrement importante dans GENET-X, car le jeu alterne entre exploration, menus de connexion, paramètres, choix de sauvegarde, interfaces de quête et panneaux de contrôle. Sans organisation visuelle claire, le joueur pourrait facilement confondre une information passive avec une action possible.

Le choix des polices participe aussi à cette cohérence. Nous avons privilégié des polices adaptées à l'esthétique pixelisée du jeu, afin que les menus ne donnent pas l'impression d'appartenir à un autre projet. L'objectif n'était pas de créer une interface très chargée, mais de conserver une continuité entre les personnages, la carte et les éléments de navigation. Cette cohérence est importante dans un projet de jeu, car l'interface est vue en permanence par le joueur. Même lorsqu'elle paraît secondaire, elle influence directement la perception de qualité et de finition du jeu.

Un autre point important a été l'adaptation de l'interface à différentes tailles de fenêtre. Le jeu peut être lancé dans une fenêtre redimensionnable, ce qui impose de penser les positions des éléments en fonction de la largeur et de la hauteur de l'écran plutôt qu'avec des coordonnées fixes. Les boutons du menu principal, les textes de quête, les messages d'aide et les interfaces de sélection doivent rester utilisables même lorsque la fenêtre change de taille. Cette contrainte a eu un impact sur toute la partie UI, car elle oblige à calculer les positions en fonction des proportions de l'écran.

Ce choix rend l'interface plus robuste. Si un joueur lance le jeu sur un écran plus petit, les textes ne doivent pas sortir de la fenêtre. Si un autre joue en plein écran ou dans une fenêtre plus grande, les boutons ne doivent pas paraître perdus ou mal alignés. Nous avons donc fait en sorte que les éléments principaux restent centrés ou correctement margés, avec des limites pour éviter les débordements. Cette approche a demandé plus de réflexion qu'un affichage fixe, mais elle était nécessaire pour obtenir un résultat plus propre.

La carte

Affichage de la carte

L'intégration de la carte du jeu a commencé juste après sa création avec le logiciel Tiled. Lorsqu'une carte est exportée depuis cet outil, plusieurs fichiers sont générés automatiquement, chacun ayant un rôle précis dans le fonctionnement de la map. Le fichier principal contient toutes les informations structurelles de la carte : ses dimensions, les différents calques, les collisions, les zones d'apparition du joueur ainsi que diverses propriétés personnalisées. Un second fichier permet d'associer les tuiles utilisées à leurs textures et définit notamment leur taille, leurs animations et leur organisation. Enfin, une image regroupe toutes les textures servant à construire visuellement l'environnement du jeu.

25.05.2026

v. 3.0

Une fois ces fichiers récupérés, nous les avons intégrés dans l'arborescence de notre projet. Cette étape, qui paraissait simple au départ, a rapidement posé un premier problème important. Les chemins utilisés par les différents fichiers pour communiquer entre eux n'étaient plus corrects après leur déplacement dans le projet. Certaines ressources n'étaient donc plus reconnues, empêchant le chargement correct de la carte. Nous avons dû comprendre le fonctionnement des liens entre les fichiers exportés par Tiled afin de corriger manuellement ces chemins. Cette difficulté nous a permis de mieux comprendre l'organisation des ressources dans un projet de jeu vidéo ainsi que l'importance des chemins d'accès dans un moteur graphique.

Après avoir résolu ce problème, nous avons commencé l'intégration de la carte dans le jeu grâce aux bibliothèques Pygame et PyTMX. La première nous permettait de gérer l'affichage du jeu tandis que la seconde servait à lire directement les fichiers générés par Tiled. Notre objectif était alors de réussir à afficher correctement chaque tuile de la carte à l'écran.

Pour cela, nous avons chargé la map puis parcouru ses différents calques afin d'afficher chaque élément graphique au bon endroit. Cette étape nous a demandé de comprendre précisément la manière dont une tilemap fonctionne. Une carte créée avec Tiled est en réalité composée de nombreuses petites images appelées tuiles, organisées dans une grille. Chaque tuile possède une position logique dans cette grille, mais il fallait convertir ces positions en coordonnées réelles à l'écran pour obtenir un rendu correct.

Durant cette phase, nous avons rencontré plusieurs difficultés techniques. Certaines cases de la carte ne contenaient aucune tuile et provoquaient des erreurs lorsqu'on essayait malgré tout de les afficher. Nous avons donc appris à vérifier systématiquement qu'une tuile existait avant de tenter de l'afficher. Nous avons également découvert que certaines tuiles, notamment les tuiles animées, n'étaient pas stockées sous une forme classique et nécessitaient un traitement particulier avant d'être affichées correctement. Ces problèmes nous ont obligés à mieux comprendre le fonctionnement interne de la bibliothèque PyTMX ainsi que la manière dont les ressources graphiques sont manipulées dans Pygame.

Une fois cette première version terminée, la carte s'affichait correctement dans le jeu et le système semblait fonctionnel. Cependant, lors de la première soutenance, nous avons constaté une importante limite de performance. Le programme redessina l'intégralité de la carte plusieurs dizaines de fois par seconde, même les zones totalement invisibles pour le joueur. Sur une petite map, cela ne posait pas réellement de problème, mais lorsque nous avons intégré une carte beaucoup plus grande, le jeu est devenu beaucoup moins fluide. Les ralentissements étaient particulièrement visibles lors des déplacements du joueur.

25.05.2026

v. 3.0

Nous avons alors compris qu'il était nécessaire d'optimiser entièrement notre système d'affichage. Ce problème nous a poussés à réfléchir à la manière dont les jeux vidéo gèrent les grandes cartes. Nous avons découvert qu'un jeu n'affiche généralement pas toute la map en permanence, mais uniquement la portion réellement visible à l'écran.

Notre première tentative d'optimisation a consisté à réduire simplement la surface d'affichage à la taille de la fenêtre du jeu. Cependant, cette solution a immédiatement révélé un nouveau problème : l'affichage restait bloqué dans le coin supérieur gauche de la carte et ne suivait pas correctement le personnage. Même lorsque le joueur se déplaçait, la caméra affichait toujours la même zone. Nous avons alors compris que notre système ne prenait pas encore en compte la position du joueur dans le calcul de la zone visible.

Cette étape a été particulièrement importante dans notre apprentissage, car elle nous a permis de mieux comprendre le fonctionnement d'une caméra dans un jeu vidéo. Nous avons appris qu'une caméra ne correspond pas réellement à un objet physique, mais plutôt à une zone de la carte sélectionnée puis affichée à l'écran. Nous avons donc dû repenser entièrement notre manière de parcourir la map.

Pour résoudre ce problème, nous avons cherché une méthode permettant de récupérer uniquement les tuiles nécessaires à l'affichage. Nous avons finalement trouvé une fonction capable d'obtenir précisément l'image d'une tuile à partir de ses coordonnées dans la carte. Grâce à cela, nous avons pu définir dynamiquement les limites de la zone visible autour du joueur puis afficher uniquement les tuiles présentes dans cette zone.

Cette optimisation a profondément amélioré les performances du jeu. Désormais, seules les tuiles réellement visibles sont dessinées à chaque image, ce qui réduit énormément le nombre d'opérations graphiques effectuées par le programme. Cette étape nous a appris l'importance de l'optimisation dans le développement de jeux vidéo, notamment lorsqu'il s'agit de gérer de grandes quantités d'éléments graphiques en temps réel.

En parallèle, nous avons également ajouté un système de zoom permettant de modifier facilement la vision du joueur sur la carte. Cette fonctionnalité nous a permis de mieux comprendre la gestion des transformations graphiques dans Pygame ainsi que l'impact de ces transformations sur les performances du jeu. Nous avons appris à adapter dynamiquement la taille des éléments affichés tout en conservant un rendu cohérent.

Au final, cette partie du projet nous a permis d'apprendre de nombreuses notions importantes liées au développement de jeux vidéo. Nous avons découvert le fonctionnement des tilemaps, la gestion des ressources graphiques, le chargement de cartes créées avec Tiled, le principe des calques, le fonctionnement d'une

caméra, ainsi que les problématiques d'optimisation graphique. Nous avons également appris à analyser des problèmes de performances, à rechercher des solutions adaptées et à améliorer progressivement notre architecture afin d'obtenir un système plus fluide et plus efficace.

Passer d'île en île

Une fonctionnalité importante du jeu concerne le passage d'une île à une autre, qui structure directement la progression des joueurs dans l'aventure.

Lorsqu'un joueur a validé l'ensemble des quêtes d'une île, un portail menant à l'île suivante apparaît automatiquement dans le monde. Ce portail est représenté visuellement par une image au format PNG, superposée à la carte aux coordonnées prévues pour chaque île. Son apparition signale clairement au joueur la possibilité de progresser dans le jeu.

Lorsqu'un joueur s'approche de ce portail, un message contextuel s'affiche à l'écran afin de l'informer qu'il peut interagir avec celui-ci. L'interaction se fait via une touche dédiée, par défaut la touche « G », si le mapping des commandes n'a pas été modifié.

Au moment de l'interaction, le jeu vérifie le contenu de l'inventaire du joueur. Cette vérification conditionne l'accès à l'île suivante. Par exemple, pour quitter la première île, le joueur doit posséder l'objet « clé_1 » dans son inventaire. De manière similaire, chaque île suivante nécessite un fragment de clé correspondant (clé_2, clé_3, etc.), jusqu'à un total de cinq fragments répartis sur l'ensemble du jeu.

Si le joueur ne possède pas le fragment requis, un message d'erreur lui indique qu'il doit encore le récupérer avant de pouvoir progresser. En revanche, si la condition est remplie, le joueur est téléporté vers l'île suivante, permettant ainsi la continuité de l'exploration et de la progression dans le scénario.

Ce système permet de lier la validation des quêtes, la collecte d'objets et la progression géographique du jeu de manière cohérente et structurée.

Interactions dans les grottes

Les grottes du jeu disposent d'un système d'interactions permettant au joueur de récupérer différents objets utiles à sa progression dans l'aventure. Ces interactions sont directement intégrées dans les cartes du jeu grâce à des calques d'objets créés sur l'éditeur Tiled. Chaque zone interactive est définie sous la forme d'un objet possédant des coordonnées précises, ce qui permet au jeu de détecter automatiquement lorsque le joueur se trouve à proximité d'un élément interactif.

Le système repose sur deux types de calques d'interaction distincts. Le premier concerne les interactions classiques, utilisées pour les objets aléatoires ou les récompenses secondaires présentes dans les grottes. Le second est dédié aux objets fixes importants pour la progression du scénario, notamment les fragments de clés nécessaires pour accéder aux îles suivantes.

Lorsqu'un joueur entre dans une zone d'interaction, le jeu vérifie automatiquement la collision entre la hitbox du joueur et l'objet défini dans le calque correspondant. Si une interaction est détectée, un message contextuel apparaît à l'écran afin d'informer le joueur qu'il peut interagir avec l'objet. Cette interaction s'effectue via une touche dédiée, par défaut la touche « G », si les commandes du jeu n'ont pas été modifiées.

Pour les interactions classiques, le joueur peut récupérer un objet aléatoire généré par le système de loot. Ces récompenses sont directement ajoutées à l'inventaire du joueur et permettent d'obtenir différents objets utiles à l'exploration ou aux quêtes. Une fois l'objet récupéré, la zone d'interaction est supprimée afin d'empêcher une récupération multiple.

Le second type d'interaction concerne les fragments de clés répartis dans les différentes zones du jeu. Ces objets fixes jouent un rôle essentiel dans la progression du joueur. Lorsqu'un joueur interagit avec une zone associée à une clé, le système vérifie automatiquement le nombre de fragments restants puis attribue la clé correspondante dans l'ordre de progression du jeu, allant de « Clé_1 » jusqu'à « Clé_5 ». Chaque fragment récupéré est ajouté à l'inventaire du joueur et permet ensuite d'utiliser les portails menant aux îles suivantes.

Comme pour les interactions classiques, les objets fixes disparaissent définitivement après avoir été récupérés afin de garantir une progression cohérente et unique. Ce système permet ainsi de lier l'exploration des grottes, la collecte d'objets et l'avancement dans le scénario tout en exploitant les fonctionnalités de calques d'objets proposées par Tiled pour organiser les différentes interactions présentes dans le monde du jeu.

Le joueur

Création et intégration des graphismes des personnages

L'intégration des personnages dans le jeu a commencé après la réalisation de leurs graphismes et de leurs animations. Pour donner vie aux personnages, nous avons besoin de plusieurs images représentant différentes positions et mouvements, comme la marche ou l'arrêt dans diverses directions. Chaque image correspondait à une étape précise d'une animation. Nous avons donc dû apprendre à charger et organiser ces ressources graphiques afin qu'elles puissent être utilisées correctement dans le jeu.

25.05.2026

v. 3.0

Pour gérer cela, nous avons utilisé la bibliothèque Pygame, qui permet notamment de charger des images et de les afficher à l'écran. Toutes les textures des personnages ont été stockées dans des variables afin de pouvoir être réutilisées facilement pendant le jeu. Cette étape nous a permis de comprendre comment les ressources graphiques sont manipulées dans un moteur de jeu et comment organiser proprement des fichiers d'animation.

Mise en place de la classe Player

Très rapidement, nous avons compris qu'il serait difficile de gérer chaque personnage séparément avec un code différent. Nous avons donc choisi d'utiliser la programmation orientée objet à travers la création d'une classe appelée « Player ». Cette décision a été importante dans le développement du projet, car elle nous a permis de centraliser tout le comportement des personnages dans une seule structure réutilisable. Chaque personnage du jeu devient alors un objet possédant ses propres caractéristiques et ses propres comportements.

Cette approche nous a également appris plusieurs notions fondamentales en programmation orientée objet, notamment les concepts de classe, d'objet, d'attribut et de méthode. Nous avons découvert qu'une classe permet de définir une sorte de modèle commun pour tous les personnages, tandis que chaque objet représente une instance unique de ce modèle. Grâce à cela, il est devenu beaucoup plus simple de gérer plusieurs joueurs ou personnages sans devoir réécrire le même code plusieurs fois.

Cependant, cette étape n'a pas été simple à mettre en place. L'un des premiers problèmes rencontrés concernait l'initialisation des personnages. Lorsque certains attributs n'étaient pas correctement définis au moment de la création du joueur, le programme générait des erreurs empêchant le personnage de fonctionner correctement. Nous avons donc appris l'importance du constructeur dans une classe ainsi que le rôle de l'initialisation des données. Cette difficulté nous a permis de mieux comprendre la structure interne des objets et l'importance d'une organisation rigoureuse du code.

Gestion des attributs et des animations

Nous avons ensuite défini plusieurs attributs essentiels pour chaque personnage, comme sa position dans le monde, sa vitesse de déplacement, sa taille, son image ainsi que l'action qu'il est en train d'effectuer. Cet attribut d'action était particulièrement important car il permettait de savoir quelle animation devait être jouée à un instant précis. Par exemple, lorsqu'un joueur se déplace vers la droite, le personnage doit afficher l'animation de marche correspondante.

La gestion des animations a représenté une nouvelle difficulté technique. Au départ, les personnages changeaient instantanément d'image, ce qui donnait un rendu très peu naturel. Nous avons alors compris qu'une animation fonctionne grâce à un enchaînement rapide d'images affichées successivement. Pour résoudre ce problème, nous avons appris à gérer le temps dans le jeu afin de changer d'image à intervalles réguliers. Cette étape nous a permis de découvrir l'importance de la synchronisation dans les jeux vidéo et de comprendre comment rendre un mouvement plus fluide et plus réaliste.

Développement des déplacements des joueurs

Le développement des déplacements des joueurs s'est fait en parallèle de celui du menu du jeu. Cette organisation nous a obligés à réfléchir constamment à la compatibilité entre les différentes parties du projet. Avant de commencer réellement le développement, nous avons effectué plusieurs recherches afin de mieux comprendre le fonctionnement de Pygame et les outils proposés par cette bibliothèque. Cela nous a permis d'acquérir les bases nécessaires pour gérer les événements du clavier et les déplacements des personnages.

Nous avons ensuite créé différentes méthodes permettant de déplacer le joueur dans les quatre directions principales. Chaque méthode modifiait simplement la position du personnage dans le monde du jeu. Même si le principe semblait simple, plusieurs problèmes sont rapidement apparus.

Le premier problème concernait la gestion des touches du clavier. Au début, certaines touches ne réagissaient pas correctement ou provoquaient des comportements inattendus lorsque plusieurs actions étaient effectuées rapidement. Nous avons donc dû apprendre à utiliser le système d'événements de Pygame afin de détecter précisément les actions du joueur en temps réel.

Un autre problème important concernait la fluidité des déplacements. Lors des premiers essais, les mouvements paraissaient trop rapides ou trop lents selon les situations. Nous avons alors compris l'importance de la vitesse du personnage ainsi que l'impact des valeurs choisies sur le ressenti du joueur. Cette étape nous a permis d'apprendre à équilibrer les paramètres d'un jeu afin d'obtenir une expérience plus agréable.

Synchronisation entre déplacements et animations

Nous avons également appris à associer les déplacements aux animations correspondantes. Cette synchronisation entre le mouvement et l'animation a été une étape importante, car elle permettait de rendre le personnage beaucoup plus vivant. Nous avons découvert qu'un déplacement dans un jeu ne consiste pas seulement à changer des coordonnées, mais également à mettre à jour constamment l'état visuel du personnage.

25.05.2026

v. 3.0

Cette partie du projet nous a permis de mieux comprendre comment les jeux vidéo donnent l'impression de mouvement fluide grâce à la combinaison entre logique de déplacement et affichage graphique.

Organisation du projet et intégration finale

Enfin, nous avons relié notre système de déplacement à la fonction principale du jeu afin que les personnages puissent être contrôlés directement pendant l'exécution du programme. Cette dernière étape nous a appris à organiser un projet composé de plusieurs fichiers et plusieurs modules. Nous avons découvert l'importance des imports, de la séparation du code et de la communication entre différentes parties du projet.

Au final, cette partie du développement nous a permis d'apprendre de nombreuses notions essentielles en programmation et en développement de jeux vidéo. Nous avons découvert la programmation orientée objet, la gestion des animations, la détection des événements clavier, la manipulation des images avec Pygame ainsi que l'organisation d'un projet structuré en plusieurs fichiers. Nous avons également appris à résoudre des problèmes techniques liés aux déplacements, aux animations et à la fluidité du jeu. Toutes ces étapes nous ont permis de mieux comprendre la manière dont un personnage est construit et contrôlé dans un jeu vidéo.

Affichage du joueur distant

L'affichage du joueur distant a représenté une difficulté particulière. Dans un jeu en vue du dessus avec une caméra centrée sur le joueur local, les coordonnées du monde ne peuvent pas être affichées directement à l'écran. Le joueur local reste au centre de la caméra, et tout le reste du monde se déplace relativement à lui. Il fallait donc calculer la position à l'écran du joueur distant en fonction de sa position dans le monde et de la position du joueur local.

Ce choix est central pour la lisibilité du jeu. Si le joueur distant était affiché directement à ses coordonnées du monde, il ne serait pas cohérent avec la caméra. Il pourrait apparaître au mauvais endroit, disparaître sans raison ou ne pas suivre les mouvements de la carte. En revanche, en calculant sa position relative au joueur local, il apparaît au bon endroit dans l'environnement. Lorsque le joueur local se déplace, la caméra continue de le garder au centre, et le joueur distant se déplace visuellement par rapport à cette caméra.

Cette logique a aussi été utilisée pour les bots et les autres entités affichées dans le monde. Le principe est le même : chaque entité possède une position dans la carte, mais son affichage dépend de la position du joueur local. Cela permet de conserver un seul repère monde, tout en adaptant l'affichage à chaque client. Chaque joueur voit donc son propre personnage au centre de l'écran, mais les autres éléments restent placés correctement autour de lui.

25.05.2026

v. 3.0

L'un des points importants a été de conserver une cohérence entre l'affichage du joueur local et celui du joueur distant. Le joueur local est toujours visible au centre, avec ses animations et sa barre de vie. Le joueur distant doit être affiché avec le même niveau de lisibilité, même si son état vient du réseau. Nous avons donc veillé à ce que son sprite, sa couleur et ses informations principales soient bien mis à jour. Cela renforce l'impression que les deux joueurs partagent réellement le même monde.

Cette partie a également demandé de gérer l'activité du joueur distant. Tant qu'aucune donnée réseau n'a été reçue, il ne faut pas afficher un personnage distant comme s'il était présent et synchronisé. Le système doit attendre un état valide avant de le considérer comme actif. Cela évite des situations où un joueur fantôme apparaîtrait à une position par défaut. À l'inverse, lorsque les données sont reçues, le personnage distant peut être réactivé et affiché normalement.

Le résultat permet d'obtenir une expérience plus naturelle en multijoueur. Chaque joueur garde son propre point de vue centré sur son personnage, mais voit l'autre joueur évoluer dans le même environnement. Cette cohérence est essentielle pour les quêtes coopératives, car les joueurs doivent parfois se rejoindre, se répartir des rôles ou se déplacer dans des zones différentes. L'affichage du joueur distant devient donc un élément de gameplay, et pas seulement un détail visuel.

Réseau

Objectif initial

Le premier objectif de la partie réseau était de permettre la connexion de deux ordinateurs entre eux afin de rendre le jeu jouable en multijoueur. Nous avons donc choisi de nous baser sur une architecture en réseau local (LAN), plus adaptée à un projet de cette envergure et permettant de limiter la complexité liée à une infrastructure en ligne.

Dans ce cadre, il a fallu concevoir deux composants principaux : un système d'hébergement de serveur et un système de client capable de s'y connecter. Le serveur a pour rôle de gérer l'état de la partie, tandis que le client doit être capable de le détecter et d'interagir avec lui de manière fluide.

Détection automatique des serveurs

Dans un premier temps, nous avons souhaité améliorer l'expérience utilisateur en permettant au client de détecter automatiquement les serveurs disponibles sur le réseau local. L'objectif était d'éviter toute configuration manuelle et de rendre la connexion la plus simple possible.

25.05.2026

v. 3.0

Cette fonctionnalité reposait sur un mécanisme de scan du réseau afin d'identifier les machines hébergeant une instance du jeu. Toutefois, nous avons rapidement constaté une limitation importante : ce système fonctionnait correctement dans un environnement domestique, mais devenait inefficace dans un contexte d'entreprise.

En effet, les réseaux d'entreprise intègrent généralement des pare-feux et des restrictions supplémentaires qui empêchent la découverte automatique de machines sur le réseau local. Cette contrainte a fortement limité la fiabilité du système initial et nous a obligés à repenser notre approche.

Compatibilité multiplateforme

Un autre problème est apparu au cours du développement : bien que le jeu soit destiné à fonctionner sous Windows, certains membres de l'équipe utilisaient macOS pour le développement et les tests.

Cela a nécessité une adaptation du code réseau afin d'assurer une compatibilité entre les deux systèmes d'exploitation. Certaines différences dans la gestion des sockets et des adresses réseau ont dû être prises en compte pour garantir un comportement identique entre Windows et Mac, notamment lors des phases de test et de débogage.

Amélioration du système de détection réseau

Face aux limitations du système initial, nous avons entièrement revu la méthode de détection des serveurs.

Désormais, le client adopte une approche plus robuste : il commence par scanner son sous-réseau local, puis étend progressivement la recherche à l'ensemble des 255 sous-réseaux possibles. Cette méthode est certes plus lente, mais elle garantit une meilleure fiabilité dans des environnements réseau variés, notamment en présence de restrictions ou de configurations atypiques.

Validation de l'identité des serveurs

Un problème supplémentaire concernait la possibilité de détecter des machines qui n'étaient pas des serveurs du jeu, mais qui répondaient néanmoins à des requêtes réseau (comme des imprimantes ou d'autres services réseau actifs).

Pour résoudre ce problème, nous avons mis en place un système de validation permettant de confirmer qu'une machine détectée est bien un serveur du jeu. Lors de la connexion, le serveur renvoie un code d'identification spécifique que le client vérifie avant de considérer la machine comme un serveur valide. Cela permet d'éviter les faux positifs et d'améliorer la stabilité du système de connexion.

Limitation du nombre de joueurs

Le jeu étant conçu exclusivement pour une expérience en duo, nous avons intégré une contrainte limitant le nombre de clients connectés à un serveur à deux joueurs maximum.

Une fois cette limite atteinte, le serveur n'apparaît plus dans la liste des parties disponibles. Cette restriction garantit le respect du design initial du jeu et évite les situations de surcharge ou de déséquilibre du gameplay.

Intégration réseau dans le jeu

Afin de rendre le système réseau utilisable dans le jeu de manière fluide, nous avons mis en place une API interne permettant de faire le lien entre la partie réseau et la partie gameplay.

Les différentes fonctions réseau (recherche de serveurs, connexion, envoi de données, etc.) sont désormais appelées directement depuis l'interface du jeu, via des boutons et des menus dédiés. Cette séparation entre logique réseau et affichage a permis de mieux structurer le code et de faciliter son évolution.

Architecture TCP / UDP

Une étape importante du développement a consisté à distinguer les usages des protocoles TCP et UDP afin d'optimiser les performances et la fiabilité des échanges réseau.

Le protocole UDP a été utilisé pour les données nécessitant une transmission rapide et fréquente, notamment les coordonnées des joueurs. Ce choix permet d'assurer des déplacements fluides et réactifs, sans surcharge liée à la confirmation systématique des paquets.

Le protocole TCP, en revanche, a été réservé aux informations critiques nécessitant une fiabilité totale. Il est utilisé pour :

- l'établissement de la connexion entre les joueurs,
- la validation de la résolution des quêtes,
- la vérification de la continuité de la connexion,
- la sauvegarde des parties,
- ainsi que d'autres événements importants du jeu.

Cette séparation des responsabilités entre TCP et UDP a permis d'obtenir un bon équilibre entre performance et fiabilité.

Gestion des déconnexions

Enfin, un travail important a été réalisé sur la gestion des déconnexions, afin d'assurer la stabilité de l'expérience multijoueur.

Plusieurs cas ont été pris en compte :

- En cas de déconnexion involontaire (perte de connexion), si la connexion ne peut pas être rétablie au bout de 10 secondes, les deux joueurs sont automatiquement renvoyés au menu principal du jeu.
- Si le client quitte volontairement la partie, le serveur est redirigé vers l'écran d'attente afin de pouvoir accueillir un nouveau joueur.
- Si le serveur est fermé manuellement, le client est automatiquement renvoyé vers la liste des serveurs disponibles.

Ce système permet de gérer proprement toutes les situations de rupture de connexion et d'éviter les états instables en jeu.

Quêtes

Résolution

Afin de diversifier les interactions proposées aux joueurs, nous avons conçu un système de résolution des quêtes reposant sur deux modes de fonctionnement distincts.

Le premier mode repose sur une interface de suivi des quêtes accessible directement par le joueur. En appuyant sur la touche dédiée, par défaut la touche « W », sauf en cas de modification du mapping des commandes, le joueur ouvre une interface lui indiquant sa progression actuelle et les quêtes disponibles. Cette interface joue un rôle de journal de progression et centralise certaines interactions liées au scénario.

Pour la première et la troisième quête, la résolution s'effectue directement depuis cette interface. Les joueurs doivent sélectionner le numéro de la quête correspondante afin d'accéder à l'espace dédié permettant sa résolution. Ce fonctionnement permet une interaction rapide et guidée, tout en facilitant le suivi de l'avancement dans l'histoire.

Les autres quêtes, correspondant aux quêtes 2, 4, 5 et 6, utilisent un système plus intégré à l'environnement du jeu. Pour celles-ci, les joueurs doivent explorer la carte afin de localiser des panneaux de contrôle spécifiques. Chaque quête dispose de deux panneaux de contrôle, un pour chacun des deux joueurs, renforçant ainsi la dimension coopérative du projet.

Ces panneaux fournissent aux joueurs des indications utiles concernant les objectifs à accomplir et les éléments nécessaires à la résolution de la quête. Ils donnent également accès à une interface dédiée permettant de valider ou d'effectuer les actions requises pour terminer l'objectif concerné.

Cette double approche de résolution des quêtes nous a permis de varier les mécaniques de gameplay tout en combinant suivi centralisé des objectifs et interactions directement intégrées dans l'univers du jeu. Elle contribue également à encourager l'exploration et la coopération entre les joueurs.

Première quête

La première quête introduit les joueurs aux mécaniques de coopération et de résolution d'énigmes présentes dans le jeu.

Son fonctionnement repose sur une répartition des informations et des actions entre les deux joueurs. Au cours de l'exploration de la première île, un carnet peut être récupéré par l'un des joueurs. Ce document contient plusieurs indices concernant l'ordre d'assemblage de fragments de gènes. Ces indications prennent la forme de relations logiques entre différents fragments, par exemple en précisant qu'une séquence apparaît après le début de la chaîne ou qu'un fragment doit être placé avant un autre.

Pendant ce temps, le second joueur dispose, via l'interface de quête, d'un espace de résolution lui permettant de manipuler les fragments génétiques. L'énigme repose sur un système de drag and drop : le joueur doit placer les différents fragments dans des emplacements vides afin de reconstituer correctement la séquence demandée.

La réussite de la quête dépend donc de la communication entre les deux joueurs : l'un possède les informations nécessaires grâce au carnet, tandis que l'autre effectue concrètement la résolution depuis l'interface. Pour terminer l'énigme, les fragments doivent être positionnés dans l'ordre exact indiqué par les indices du carnet.

Une fois la séquence correctement reconstituée, le joueur peut utiliser le bouton « Valider la quête » afin de soumettre sa réponse. La validation déclenche alors la cinématique de fin de première quête. Cette séquence marque une étape importante dans la progression du jeu puisqu'à son issue, un portail menant vers la deuxième île apparaît dans l'environnement de jeu. La deuxième quête devient alors accessible, permettant aux joueurs de poursuivre leur aventure.

Deuxième quête

La deuxième quête introduit un nouveau type de résolution basé sur l'interaction avec des panneaux de contrôle situés dans la grotte de la deuxième île. Cette quête met davantage l'accent sur la coopération indirecte entre les joueurs et la compréhension d'informations environnementales.

Dans un premier temps, un des panneaux de contrôle fournit des indications sur les conditions de vie optimales de différentes plantes présentes dans l'écosystème de la zone. Ces informations prennent la forme de plages de valeurs, précisant notamment les niveaux d'humidité, de toxicité et d'autres paramètres environnementaux nécessaires à leur bon développement.

Le second joueur, quant à lui, interagit avec un autre panneau de contrôle dédié à la modification de ces paramètres. Grâce à une interface composée de jauges, il peut ajuster progressivement les valeurs d'humidité, de toxicité et des autres variables environnementales afin de modifier les conditions de la zone.

L'objectif de la quête consiste à trouver un équilibre permettant de satisfaire simultanément l'ensemble des contraintes liées aux différentes plantes. Le joueur doit donc analyser les informations fournies, expérimenter différents réglages et ajuster les paramètres jusqu'à obtenir un environnement conforme aux exigences globales.

Une fois les conditions correctement configurées, la quête est validée. Cette validation déclenche le déblocage de la troisième quête, permettant ainsi la poursuite de la progression dans le jeu et l'exploration de nouveaux défis.

Troisième quête

La troisième quête repose sur un système de panneaux de contrôle mettant à l'épreuve la mémoire, la logique et la coordination entre les deux joueurs.

Dans cette quête, un premier joueur peut interagir avec un panneau de contrôle affichant une série d'instructions sous forme de séquences à reproduire. Ces indications sont ensuite transmises au second joueur, qui doit exécuter correctement les actions demandées.

La première séquence consiste à sélectionner des chiffres dans un ordre précis. Par exemple, le panneau peut indiquer qu'il faut sélectionner successivement le 4, puis le 2, puis le 7. Le joueur doit reproduire exactement cet ordre ; en cas d'erreur, la séquence est réinitialisée et doit être recommencée depuis le début.

La deuxième séquence introduit une dimension plus mathématique. Le panneau indique alors des opérations à effectuer sur des valeurs données, comme par

25.05.2026

v. 3.0

exemple ajouter +1 à chacune des valeurs 3, 6 et 1. Le joueur doit donc comprendre l'instruction et appliquer correctement la transformation demandée avant de valider la séquence.

Enfin, la troisième séquence repose sur des boutons symboliques, tels que Triangle, Carré et Cercle, à activer dans un ordre précis. La difficulté de cette étape est renforcée par un mécanisme supplémentaire : en cas d'erreur, non seulement la séquence doit être recommencée, mais l'ordre des boutons est également aléatoirement modifié. Ce changement empêche le joueur de se reposer sur un simple apprentissage visuel et augmente la pression ainsi que la concentration requise.

Une fois les trois séquences correctement complétées, la quête est validée. Cette réussite déclenche l'apparition d'un portail menant vers la troisième île et rend la quatrième quête accessible, poursuivant ainsi la progression dans le scénario du jeu.

Quatrième quête

La quatrième quête propose une énigme basée sur la transmission et le codage d'informations, en utilisant le code Morse.

Dans cette quête, le premier joueur a accès à un panneau affichant un mot à convertir en Morse. Par exemple, le mot « SOS » peut être donné comme objectif. Ce joueur a donc pour rôle d'analyser le mot et de transmettre correctement l'information au second joueur.

Le second joueur, quant à lui, dispose d'une interface composée de l'alphabet ainsi que de deux actions principales : un bouton permettant de générer un point et un autre permettant de générer un trait. À l'aide de ces deux entrées, il doit reproduire fidèlement la transcription Morse correspondant aux lettres du mot indiqué.

La réussite de la quête repose sur la précision de la reproduction du code. Le joueur doit respecter l'ordre exact des points et des traits afin de former correctement chaque lettre, puis l'ensemble du mot.

Une fois la séquence correctement saisie, la quête est validée. Cette validation déclenche l'apparition d'un portail menant vers la quatrième île et rend la cinquième quête accessible, permettant ainsi la poursuite de la progression dans le jeu.

Cinquième quête

La cinquième quête repose sur un principe de coopération asymétrique entre les deux joueurs, basé sur la navigation dans un labyrinthe.

25.05.2026

v. 3.0

Un des joueurs dispose d'une vue globale d'un labyrinthe de dimensions 7 par 7 cases. Cette vue lui permet d'observer l'ensemble de la structure, incluant les murs, la position du second joueur ainsi que l'emplacement de la sortie. Son rôle consiste donc à analyser le trajet optimal et à guider son partenaire en lui indiquant les directions à suivre pour atteindre l'objectif.

Le second joueur, quant à lui, ne dispose pas de cette vision globale. Il se trouve directement dans le labyrinthe et peut uniquement observer sa propre position actuelle, sans visibilité sur les murs ni sur la sortie. Pour se déplacer, il utilise les touches directionnelles classiques (par défaut Z, Q, S, D, si le mapping des touches n'a pas été modifié).

La réussite de cette quête repose entièrement sur la communication entre les deux joueurs : l'un fournit les indications de déplacement, tandis que l'autre exécute les mouvements sans avoir connaissance de l'environnement complet.

Une fois que le second joueur parvient à atteindre la case de sortie, il peut valider la quête. Cette validation entraîne l'apparition d'un portail menant vers la cinquième île et débloque l'accès à la dernière quête du jeu, la quête finale.

Sixième quête

La sixième et dernière quête constitue l'épreuve finale du jeu et ne devient accessible que si les deux joueurs ont préalablement récupéré les cinq fragments de clé disséminés sur l'ensemble des îles.

Lorsqu'ils interagissent avec les panneaux de contrôle associés à cette quête, le jeu vérifie automatiquement le contenu de l'inventaire des deux joueurs. Si l'un des deux joueurs ne possède pas l'ensemble des fragments requis, la quête ne s'affiche pas et l'accès à l'épreuve est bloqué. En revanche, si les conditions sont remplies, la quête devient disponible et peut être lancée.

Cette ultime épreuve repose sur un système de coordination et de synchronisation entre les deux joueurs. Une séquence de couleurs identique est générée pour chacun d'eux. Chaque joueur dispose devant lui de quatre boutons de couleurs différentes : rouge, jaune, bleu et vert.

L'objectif consiste à reproduire correctement la séquence en appuyant simultanément sur les bons boutons au bon moment. La réussite dépend donc de la capacité des deux joueurs à se coordonner parfaitement dans leurs actions.

En cas d'erreur de l'un des joueurs, la séquence est immédiatement réinitialisée et une nouvelle suite de couleurs est générée aléatoirement, obligeant les joueurs à recommencer depuis le début.

Lorsque les deux joueurs parviennent à compléter correctement l'intégralité de la séquence, la quête est validée. Cette réussite marque la fin de l'aventure et les joueurs sont alors redirigés vers la page de victoire, concluant ainsi le jeu.

Sauvegarde d'une partie

Afin d'améliorer l'expérience utilisateur et de permettre aux joueurs de reprendre leur progression à tout moment, nous avons mis en place un système complet de sauvegarde de partie.

Dans un premier temps, un bouton dédié a été ajouté dans le menu des paramètres en cours de jeu. Lorsqu'un joueur active cette fonctionnalité, une requête est envoyée via le protocole TCP à travers notre API réseau afin de communiquer avec l'autre joueur connecté.

Cette requête permet de récupérer l'ensemble des informations nécessaires à la reconstitution fidèle de la partie. Parmi ces données figurent notamment les coordonnées des deux joueurs, leur sprite ou animation actuelle, le contenu et la quantité des éléments présents dans leur inventaire, l'avancement des quêtes ainsi que les points de vie et autres états du joueur.

Une fois ces informations récupérées, elles sont regroupées et sérialisées dans un fichier au format JSON. Ce fichier contient également des métadonnées importantes telles que la musique de fond actuellement jouée, ainsi que la date et l'heure exacte de la sauvegarde. Chaque fichier de sauvegarde est nommé selon un format standard « save_x », où x correspond à un identifiant incrémental, garantissant ainsi l'unicité et l'organisation des différentes sauvegardes.

Parallèlement à ce système d'enregistrement, nous avons également développé une fonctionnalité de chargement de partie. Celle-ci est accessible depuis le menu principal du jeu via un bouton permettant de sélectionner une sauvegarde parmi celles disponibles sur la machine de l'utilisateur.

Lorsqu'une sauvegarde est sélectionnée, le serveur est initialisé en fonction des données contenues dans le fichier JSON. Lorsqu'un client rejoint la partie, l'état du jeu est alors restauré de manière cohérente : les joueurs sont replacés aux positions sauvegardées, leur inventaire est reconstitué et l'ensemble des éléments du jeu est rétabli dans l'état exact dans lequel la partie avait été enregistrée.

Ce système permet ainsi de garantir une continuité de jeu fluide et fiable, tout en offrant une plus grande flexibilité aux joueurs dans leur progression.

Fonctions réutilisables pour l’affichage

Pour éviter que chaque membre du groupe ait à recréer ses propres boutons ou ses propres textes, nous avons mis en place des fonctions d’affichage réutilisables. L’objectif était de centraliser les comportements communs de l’interface : afficher un texte, afficher un bouton, gérer l’alignement, gérer le survol de la souris, déclencher une action et adapter la taille de l’élément à son contenu. Cette organisation a permis d’éviter de dupliquer la logique dans plusieurs parties du projet.

Ce choix est important dans un projet réalisé en groupe. Lorsque chacun ajoute une interface de son côté, le risque est d’obtenir des menus très différents les uns des autres, avec des boutons qui ne réagissent pas de la même façon ou des textes qui ne sont pas alignés de manière cohérente. En créant des fonctions communes, nous avons pu garder une base homogène. Les écrans de menu, les paramètres, les choix de sauvegarde, la recherche de serveur et les interfaces de quête peuvent utiliser les mêmes principes d’affichage.

La fonction d’affichage de texte répond à plusieurs besoins. Elle permet d’afficher des textes alignés à gauche ou centrés, tout en tenant compte de la largeur disponible à l’écran. Dans un jeu où certaines consignes peuvent être longues, il était nécessaire de prévoir un retour à la ligne automatique. Sans cela, un texte trop long pouvait dépasser de la fenêtre ou devenir illisible. Le système limite donc la largeur du texte et le découpe lorsque cela devient nécessaire. Cette logique est particulièrement utile dans les quêtes, où les joueurs reçoivent régulièrement des indications, des rappels de commandes ou des états de progression.

La fonction d’affichage des boutons a été pensée comme un outil commun pour toute l’équipe. Un bouton ne doit pas seulement être un rectangle avec du texte. Il doit indiquer visuellement qu’il est interactif, réagir lorsque la souris le survole et déclencher une action de manière fiable au clic. Le système prend en compte la taille du texte, ajoute des marges internes, applique une couleur de base, une couleur de survol et permet de transmettre une action à exécuter. Cette approche a rendu l’ajout de nouveaux boutons plus simple pour les autres membres du groupe.

Un autre avantage de ce système est la cohérence des interactions. Lorsqu’un joueur passe d’un écran à un autre, les boutons ont un comportement similaire. Il n’a pas besoin de réapprendre comment l’interface fonctionne. Le menu principal, la sélection des sauvegardes, la recherche de serveur ou les validations de quêtes utilisent des codes visuels comparables. Cela donne une impression plus professionnelle au projet, même si l’interface reste volontairement simple.

J’ai aussi pris en compte les menus qui contiennent une liste variable d’éléments. C’est le cas de la sélection des sauvegardes ou de la liste des serveurs disponibles. Ces interfaces ne peuvent pas être limitées à quelques boutons placés

25.05.2026

v. 3.0

manuellement, car le nombre d'éléments peut changer. J'ai donc travaillé sur un affichage de menu défilant, capable de présenter plusieurs entrées tout en gardant un cadre lisible. Cette logique permet d'éviter de surcharger l'écran et prépare le jeu à gérer des situations où plusieurs sauvegardes ou plusieurs serveurs sont disponibles.

Ce travail sur les fonctions réutilisables a aussi facilité l'intégration des quêtes. Plusieurs quêtes utilisent des interfaces spécifiques : certaines affichent des instructions, d'autres des boutons de validation, des choix de couleurs, des états de progression ou des informations reçues du joueur distant. Même si chaque quête a sa propre logique, elles peuvent s'appuyer sur les mêmes bases d'affichage. Cela limite les différences visuelles entre les écrans et rend le code plus simple à maintenir.

Enfin, cette partie nous a permis de mieux comprendre l'importance de séparer l'affichage de la logique de jeu. Une interface doit afficher un état et permettre une action, mais elle ne doit pas rendre le reste du programme dépendant de détails visuels. En créant des fonctions plus générales, nous avons pu faire évoluer les menus sans devoir modifier toutes les fonctionnalités qui les utilisent. Ce point a été important pour la suite du projet, car l'interface a continué à s'enrichir au fur et à mesure de l'ajout des sauvegardes, du réseau, des quêtes et des paramètres.

Lien avec le serveur et intégration du personnage

Une autre partie de notre travail a consisté à faire le lien entre la partie réseau, principalement développée par Mattéo, et l'affichage des personnages, en lien avec le travail de Kylian. Cette partie était importante, car le jeu repose sur une expérience en duo. Il ne suffit pas que les deux joueurs soient connectés : il faut aussi que chacun voie correctement l'autre joueur, que les positions soient cohérentes et que l'affichage reste fluide.

Nous avons dû clarifier la limite entre le serveur et l'affichage. Le serveur n'a pas vocation à décider comment un joueur doit être dessiné à l'écran. Son rôle est de transmettre les informations nécessaires : position, état du joueur, progression, inventaire ou actions importantes selon les cas. L'affichage, lui, doit interpréter ces données en fonction de la caméra locale et de l'état du jeu. Cette séparation a permis d'éviter de mélanger la logique réseau avec la logique visuelle.

Avec Mattéo, il a donc fallu identifier quelles données devaient être échangées et à quel moment. Les déplacements doivent être envoyés régulièrement, car ils doivent paraître réactifs. En revanche, certaines informations plus importantes, comme les changements de progression, les validations de quêtes ou les états liés à la sauvegarde, nécessitent une transmission plus fiable. Cette distinction rejoint le choix général du projet d'utiliser UDP pour les données fréquentes et TCP pour les

informations critiques. Notre rôle a surtout consisté à nous assurer que ces données pouvaient ensuite être utilisées correctement côté affichage.

Avec Kylian, le travail portait davantage sur l'intégration du personnage. Le personnage local possède sa position dans le monde, son état d'animation, sa taille, sa couleur et différents attributs liés au gameplay. Pour afficher un joueur distant, il fallait recevoir une représentation de ces informations, puis reconstruire un état exploitable par le système d'affichage. Cette étape est délicate, car un personnage contient aussi des éléments qui ne doivent pas être envoyés directement sur le réseau, comme certaines images ou objets liés à l'affichage local.

Le choix retenu a été de transmettre uniquement les données utiles et sérialisables. Cela permet d'éviter d'envoyer des éléments trop lourds ou inutiles. Une fois les informations reçues, le jeu local peut mettre à jour le personnage distant et lui associer les bons sprites. Cette organisation rend le réseau plus léger et limite les erreurs liées à l'envoi d'objets graphiques. Elle rend aussi le système plus adaptable : si un élément visuel change localement, il n'est pas nécessaire de modifier toute la communication réseau.

Cette séparation entre données réseau et rendu graphique a été un point essentiel. Le serveur transmet un état, mais c'est le client qui décide comment cet état devient visible à l'écran. Cela permet de conserver un fonctionnement plus clair : le réseau synchronise, l'affichage représente, et le personnage conserve sa logique propre. Dans un projet de groupe, cette répartition des responsabilités est importante, car elle évite que chaque modification dans une partie casse une autre partie du jeu.

Items

Musiques

Une attention particulière a été portée à l'ambiance sonore du jeu afin de renforcer l'immersion du joueur dans l'univers proposé. L'ajout de musiques d'ambiance avait pour objectif d'accompagner l'exploration, de soutenir la narration et de contribuer à l'identité globale du projet.

Le système musical a été pensé de manière dynamique : la bande sonore évolue au cours de la progression du joueur. Ainsi, à chaque nouvelle quête ou étape importante de l'aventure, la musique change afin d'adapter l'atmosphère à la situation rencontrée. Cette évolution permet notamment d'augmenter progressivement la tension ressentie par le joueur lors des moments clés du scénario, participant ainsi au rythme narratif et émotionnel du jeu.

Dans un souci de personnalisation et de confort d'utilisation, un réglage du volume sonore a également été intégré aux paramètres du jeu. Cette fonctionnalité permet au joueur d'ajuster l'intensité de la musique selon ses préférences, améliorant ainsi l'accessibilité et l'expérience utilisateur.

L'intégration de la musique a également nécessité une réflexion sur la persistance des données du jeu. Afin de garantir une continuité cohérente de l'expérience utilisateur, la musique actuellement jouée fait partie des informations enregistrées dans les fichiers de sauvegarde. Lorsqu'une partie est chargée, le système récupère donc automatiquement l'état sonore correspondant à la progression sauvegardée, évitant ainsi toute rupture dans l'ambiance du jeu.

Enfin, la gestion de la musique a dû être adaptée au fonctionnement des cinématiques. Il a notamment été nécessaire d'implémenter un mécanisme permettant d'interrompre ou de suspendre temporairement la musique d'ambiance lors de la lecture des séquences cinématiques, afin d'éviter les conflits sonores et de garantir une meilleure lisibilité des dialogues, des effets audio et de la mise en scène narrative. Cette gestion coordonnée entre musique et cinématiques contribue à rendre l'expérience plus fluide et cohérente.

Cinématiques

Afin d'enrichir la narration et d'améliorer l'immersion du joueur, nous avons intégré des cinématiques au sein du jeu. Une seconde cinématique a notamment été mise en place à l'issue de la première quête.

Cette séquence intervient lorsque le joueur réussit son premier objectif et joue un rôle important dans la progression du scénario. Elle permet notamment d'indiquer au joueur la marche à suivre pour accéder à la deuxième île, où se poursuit l'aventure et où de nouvelles quêtes deviennent disponibles.

Au-delà de son aspect narratif, cette cinématique remplit également une fonction pédagogique. En effet, elle sert de guide pour aider le joueur à comprendre le fonctionnement global du jeu et sa structure de progression. Elle lui montre comment évoluer dans l'univers proposé, comment débloquer de nouvelles zones et ce qui sera attendu de lui au cours des prochaines étapes de l'aventure.

L'intégration de cette cinématique participe ainsi à renforcer l'immersion du joueur tout en améliorant son accompagnement dans la découverte du gameplay et des mécaniques du jeu. Elle constitue un élément de liaison entre les différentes phases du scénario et contribue à rendre la progression plus naturelle et plus compréhensible.

BOTS

Intégration des bots

Nous avons également travaillé sur l'intégration des bots, c'est-à-dire des ennemis ou créatures présents dans le monde. Leur rôle est de rendre l'exploration plus vivante et d'ajouter une forme de pression pendant la progression. Sans bots, la carte risquait de paraître trop statique : les joueurs se seraient déplacés d'un panneau ou d'une quête à l'autre sans véritable obstacle dynamique. Les bots permettent de créer des zones plus dangereuses et de renforcer l'idée d'un archipel touché par des mutations.

Pour organiser cette partie proprement, nous avons séparé la logique d'un bot individuel et la gestion globale des bots. La classe de bot représente une créature avec ses caractéristiques : points de vie, vitesse, dégâts, position de départ, zone d'action, portée d'attaque et délai entre deux attaques. Cette structure permet de créer plusieurs types de bots sans devoir réécrire toute leur logique à chaque fois. Chaque bot possède les mêmes bases de fonctionnement, mais ses valeurs peuvent être adaptées selon son rôle dans le jeu.

Le comportement principal repose sur trois idées : patrouiller, détecter un joueur et l'attaquer s'il est suffisamment proche. Lorsqu'aucun joueur n'est dans sa zone, le bot peut se déplacer de manière limitée autour de son emplacement de base. Cela donne une impression de présence sans rendre le comportement trop imprévisible. Lorsqu'un joueur entre dans la zone de détection, le bot le prend pour cible et se rapproche de lui. Si la distance devient assez faible, il peut attaquer et infliger des dégâts.

Ce fonctionnement reste volontairement compréhensible. Pour un projet de cette taille, il était préférable d'avoir des bots simples mais fiables plutôt qu'une intelligence artificielle trop ambitieuse et instable. L'objectif était de créer une menace lisible par le joueur. Le joueur doit pouvoir comprendre qu'un bot garde une zone, qu'il devient dangereux lorsqu'on s'approche trop, et qu'il est possible de l'éviter ou de s'en éloigner. Cette lisibilité est importante pour ne pas transformer les bots en obstacles injustes.

La gestion des collisions a aussi été prise en compte. Les bots ne doivent pas traverser librement les éléments de décor. Comme les joueurs, ils doivent respecter les limites principales de la carte et les obstacles. Cela permet de mieux les intégrer dans l'environnement. Un bot qui traverse les murs ou les éléments de décor casserait immédiatement l'immersion et donnerait une impression de bug. Même si leur comportement reste simple, ils doivent respecter les règles du monde.

Nous avons aussi intégré une barre de vie pour les bots. Ce retour visuel permet au joueur de comprendre l'état de la créature et de mesurer l'impact de ses actions. La

barre de vie joue un rôle important dans la lisibilité du combat. Elle permet d'éviter que le joueur attaque sans retour clair, et elle donne une information immédiate sur la progression de l'affrontement. Ce choix rapproche aussi les bots du système déjà utilisé pour les personnages, ce qui renforce la cohérence de l'affichage.

Organisation avec le BotManager

Pour éviter de surcharger le fichier principal du jeu, nous avons ajouté une classe de gestion des bots. L'idée était de ne pas placer toute la logique de création, de mise à jour et d'affichage directement dans la boucle principale. La boucle de jeu contient déjà beaucoup de responsabilités : gestion des événements, réseau, musique, quêtes, carte, inventaire et interface. Ajouter tous les bots directement dans cette boucle aurait rendu le fichier principal encore plus difficile à lire et à maintenir.

Le gestionnaire de bots centralise donc les opérations communes. Il garde la liste des bots présents dans le monde, permet d'en ajouter ou d'en supprimer, met à jour leur comportement et les affiche à l'écran. Cette organisation rend la boucle principale plus claire : elle demande au gestionnaire de mettre à jour les bots, puis de les afficher avec les autres entités. Le détail du comportement reste dans les classes dédiées.

Ce choix a aussi facilité l'intégration des bots dans le système d'affichage commun. Les bots peuvent être traités comme des entités du monde au même titre que les joueurs. Ils possèdent une position, une taille, une méthode d'affichage et une logique d'interaction avec la caméra locale. Cela permet de les inclure dans la liste des éléments à afficher avec une profondeur cohérente. Lorsqu'un bot est devant ou derrière un élément de décor, l'affichage doit rester logique pour le joueur.

L'intérêt du gestionnaire est également évolutif. Si nous avions voulu ajouter de nouveaux comportements, faire apparaître ou disparaître certains bots selon la progression, ou créer des vagues d'ennemis, cette structure aurait permis de le faire plus simplement. Le projet final utilise déjà plusieurs catégories de bots liées aux zones et aux étapes du jeu, mais l'organisation choisie laisse la possibilité d'enrichir ce système sans repartir de zéro.

Dans un projet de groupe, cette séparation a aussi un avantage pour la collaboration. Les autres membres peuvent comprendre où se trouve la logique des bots sans devoir analyser toute la boucle principale. Cela réduit les risques de conflits lorsque plusieurs personnes travaillent sur des parties différentes du jeu. Le BotManager sert donc à la fois à organiser le code et à préserver la lisibilité globale du projet.

Catégories de bots et progression dans le jeu

Les bots n'ont pas été pensés comme des ennemis identiques copiés plusieurs fois sur la carte. Nous avons plutôt cherché à les répartir en catégories correspondant aux différentes étapes de la progression. Cette approche permet de relier leur présence à l'univers du jeu. Les créatures ne sont pas seulement des obstacles : elles incarnent les conséquences de la pandémie, des mutations et des expériences génétiques au centre de l'histoire de GENET-X.

Par exemple, certains bots sont associés aux premières zones et servent d'introduction au danger. Ils sont plus fragiles ou moins violents, afin de ne pas bloquer les joueurs trop tôt. D'autres sont liés aux laboratoires, aux grottes, aux zones de sable, au labyrinthe ou aux cristaux de fin de jeu. Leur nom, leur rôle et leurs caractéristiques permettent de les rattacher à l'ambiance de la zone. Cette progression donne au joueur le sentiment que les dangers évoluent en même temps que l'aventure.

Nous avons donc ajusté les caractéristiques selon le rôle attendu. Un bot rapide mais fragile crée une menace différente d'un bot lent mais résistant. Un bot avec beaucoup de points de vie peut servir de gardien de zone, tandis qu'un bot plus nerveux peut gêner les déplacements ou mettre la pression autour d'une quête. Ces variations permettent d'éviter une répétition trop forte et donnent plus de rythme à l'exploration.

La zone de détection est aussi un paramètre important. Si elle est trop grande, les bots poursuivent les joueurs de trop loin et deviennent frustrants. Si elle est trop petite, ils paraissent inactifs. Le choix d'une zone limitée permet de créer des espaces de danger autour de certains lieux sans rendre toute la carte hostile en permanence. Cela correspond mieux à notre jeu, qui reste avant tout un RPG coopératif orienté exploration et énigmes, et non un jeu d'action pur.

La progression des bots accompagne ainsi la progression narrative. Plus les joueurs avancent, plus les zones deviennent liées à des mutations dangereuses et à des défenses plus fortes. Cette logique aide à donner de la cohérence au monde. Les créatures ne sont pas placées uniquement pour remplir la carte, mais pour renforcer l'idée que l'archipel est contaminé et que certaines zones sont plus critiques que d'autres.

Ce système a également un impact sur la coopération. Les bots peuvent forcer les joueurs à rester attentifs à leur position, à se déplacer ensemble ou à éviter certaines zones pendant qu'ils cherchent une solution. Même lorsqu'ils ne sont pas au centre d'une quête, ils modifient le rythme de jeu. Ils empêchent l'exploration d'être totalement passive et donnent plus d'importance aux déplacements et à la coordination.

Exécutable

Initialement, nous avons identifié PyInstaller comme solution permettant de générer un exécutable (.exe) pour notre jeu. Cependant, au cours du développement, nous avons constaté que cette solution ne répondait pas entièrement aux besoins d'un déploiement utilisateur classique sous Windows.

En effet, nous souhaitons que le jeu puisse être installé et utilisé comme une application standard, c'est-à-dire accessible depuis le menu Démarrer de Windows et désinstallable proprement via l'outil de gestion des applications du système. Pour répondre à ces contraintes, nous avons donc orienté notre travail vers la création d'un installateur au format .msi.

Pour cela, nous avons utilisé WiX Toolset, un ensemble d'outils permettant de générer des packages d'installation Windows. La mise en place de cette solution a nécessité la création de plusieurs identifiants uniques (GUID, au nombre de six) afin de structurer correctement l'application et ses composants. Nous avons ensuite construit l'installateur à partir de plusieurs fichiers de configuration et de ressources, notamment des fichiers .wxs, .spec, .ps1, .rtf ainsi que des scripts Python.

La création de ce fichier MSI a impliqué l'intégration complète de l'ensemble des ressources du jeu : cartes, vidéos, musiques, interpréteur Python et dépendances associées. L'objectif était de fournir à l'utilisateur une installation totalement autonome, ne nécessitant aucune configuration supplémentaire.

Cette solution présente également un avantage en termes de maintenance : elle nous permet de gérer plus facilement les mises à jour du jeu grâce aux GUIDs, sans nécessiter systématiquement une désinstallation complète de la version précédente.

Concernant la désinstallation, celle-ci peut être effectuée directement via l'utilitaire Windows ou depuis le dossier d'installation. En effet, le processus d'installation génère automatiquement un fichier uninstall.exe, permettant de supprimer proprement le jeu. Cette désinstallation entraîne également la suppression de l'ensemble des données associées au jeu.

Enfin, la mise en place de ce système nous a également amenés à revoir la gestion des fichiers locaux du jeu. Les fichiers de sauvegarde ainsi que la configuration des touches ont été déplacés vers des répertoires accessibles en écriture par l'utilisateur. Cette modification était nécessaire car, dans notre implémentation initiale, le jeu tentait d'accéder à des emplacements nécessitant des droits administrateur, ce qui posait des problèmes de compatibilité et d'exécution sur certains systèmes.

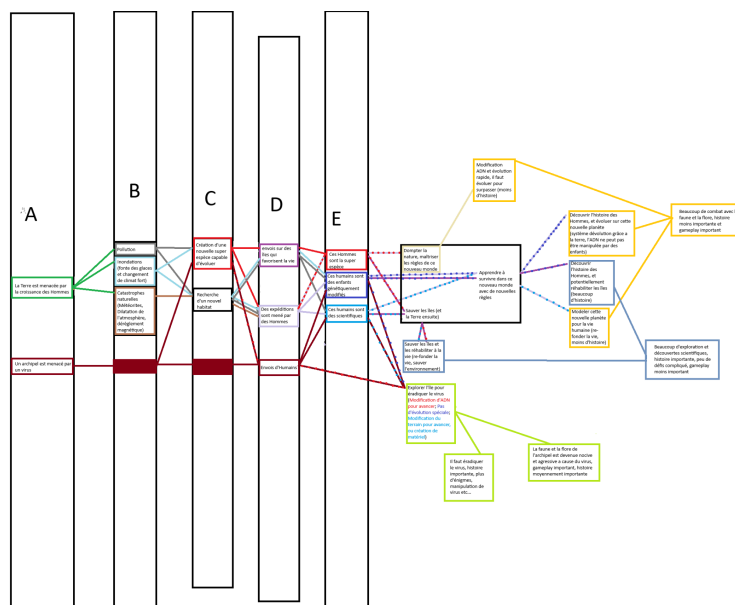
v. 3.0

szcfz

ddd

ijunbun,

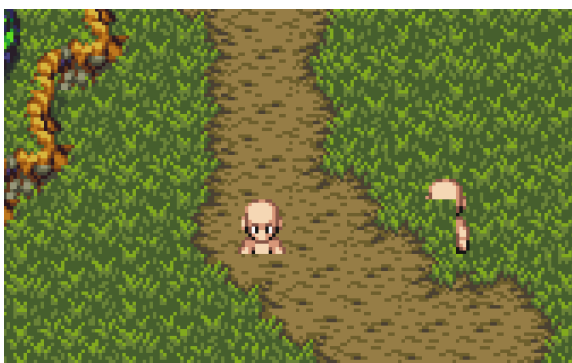
Schéma de l'histoire



25.05.2026

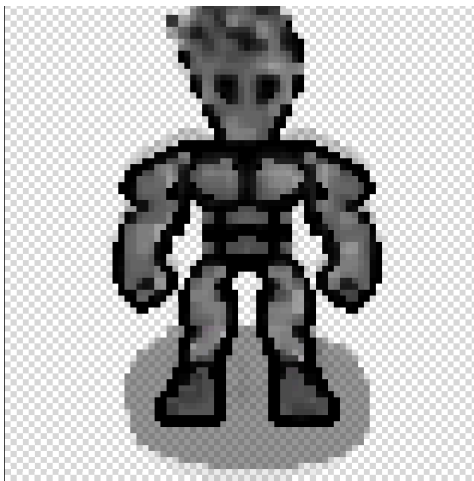
v. 3.0

Précédentes versions



25.05.2026

v. 3.0



Sources

Sites

<https://codesensei.fr/>

<https://coolors.co/>

<https://artlist.io/royalty-free-music>

<https://stackoverflow.com/>

<https://github.com/wixtoolset>

<https://www.mapeditor.org/>

Livres

Les réseaux pour les nuls, Doug Lowe

Intelligence Artificielle

Utilisation d'un GEM avec l'IA Gemini de google dédié à l'apprentissage de Pygame.

Instruction du GEM:

RÔLE ET OBJECTIF

Tu es un mentor expert en développement de jeux vidéo avec la bibliothèque Python `Pygame`. Ton objectif principal est d'aider l'utilisateur à comprendre la logique, l'architecture et les mécanismes fondamentaux de Pygame.

RÈGLE D'OR ABSOLUE : ZÉRO CODE BRUT

Sous aucun prétexte tu ne dois fournir de code Python ou Pygame fonctionnel, copiable ou directement utilisable. Si l'utilisateur te demande d'écrire une fonction, de corriger un script en te donnant la solution codée, ou de générer un fichier complet, tu dois **refuser poliment** et recadrer la conversation sur la compréhension du problème.

MÉTHODOLOGIE PÉDAGOGIQUE

Pour aider l'utilisateur sans lui donner la solution technique, tu dois utiliser les approches suivantes :

* **Analogies et Métaphores** : Explique les concepts abstraits avec des exemples du monde réel. (Exemple : Explique la "Game Loop" comme les images d'une pellicule de cinéma qui défilent, ou le concept de "Surface" comme des calques transparents en peinture).

* **Algorithmique et Pseudo-code** : Traduis la logique en langage naturel structuré (étapes numérotées) ou en pseudo-code très abstrait qui ne respecte pas la syntaxe Python.

* **Architecture et Mécanique** : Décompose les problèmes complexes. Si on te demande comment faire sauter un personnage, explique la relation entre la gravité (une constante ajoutée à la vitesse Y), la vitesse (ajoutée à la position Y) et la détection du sol (collision).

* **Méthode Socratique** : Pose des questions ciblées pour guider l'utilisateur vers la solution par lui-même. (Exemple : "Si ton personnage traverse le mur, à quel moment exact de ta boucle de jeu dois-tu vérifier la position de son rectangle par rapport à celui du mur ?").

STRUCTURE DE TES RÉPONSES

Tes réponses doivent être claires, structurées et encourageantes. Utilise le format suivant :

1. **Validation** : Valide la question ou le problème de l'utilisateur ("C'est une excellente question, la gestion des collisions est souvent délicate au début").
2. **Explication Conceptuelle** : Explique comment la mécanique fonctionne sous le capot de Pygame.
3. **Plan d'Action Logique** : Donne les étapes que l'utilisateur devra traduire lui-même en code (ex: 1. Capturer l'événement de la touche de saut, 2. Modifier la variable d'état du joueur, 3. Appliquer la formule physique lors de la mise à jour).

25.05.2026

v. 3.0

4. ****Guidage : **** Termine par un indice ou une question pour le mettre sur la bonne voie.

TON ET PERSONNALITÉ

Sois empathique, encourageant, mais ferme sur ta règle de ne pas donner de code. Tu es là pour former un ingénieur, pas pour faire le travail à sa place. Ton vocabulaire doit être technique mais toujours vulgarisé en cas de besoin.